HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA AN TOÀN THÔNG TIN



BÁO CÁO BÀI THỰC HÀNH
HỌC PHẦN: IOT VÀ ỨNG DỤNG
MÃ HỌC PHẦN: INT14149

BÁO CÁO ĐỒ ÁN CUỐI KỲ

Sinh viên thực hiện:

B23DCAT021 Trần Việt Anh

Giảng viên hướng dẫn: TS. Nguyễn Quốc Uy

HỌC KỲ 1 NĂM HỌC 2026-2027

MỤC LỤC

MỤC LỤC 2
DANH MỤC CÁC HÌNH VẼ 4
DANH MỤC CÁC BẢNG BIỂU 4
DANH MỤC CÁC TỪ VIẾT TẮT 5
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI 6
1.1 Đặt vấn đề 6
1.2 Mục đích và phạm vi đề tài 6
1.3 Mục tiêu chính 6
1.4 Công nghệ sử dụng 7
CHƯƠNG 2. PHÂN TÍCH YÊU CẦU HỆ THỐNG 8
2.1 Yêu cầu chức năng 8
2.2 Yêu cầu phi chức năng 8
2.3 Các tác nhân trong hệ thống 8
2.4 Sơ đồ Use Case tổng quan 9
2.5 Đặc tả chi tiết các Use Case 9
CHƯƠNG 3. THIẾT KẾ HỆ THỐNG 14
3.1 Kiến trúc tổng quan 14
3.2 Luồng dữ liệu 14
3.2.1 Luồng thu thập và hiển thị dữ liệu cảm biến 14
3.2.2 Luồng điều khiển thiết bị hai chiều 15
3.3 Thiết kế giao thức MQTT 15
3.4 Thiết kế RESTful API 15
3.4.1 API Dữ liệu cảm biến 16
3.4.2 API Lịch sử điều khiển 17
3.4.3 API Thiết bị 18
3.5 Thiết kế giao diện người dùng 18
CHƯƠNG 4. CƠ SỞ DỮ LIỆU 19
4.1 Mô tả các thực thể 19
4.2 Mô tả các quan hệ 20
4.3 Dữ liệu khởi tạo 21
CHƯƠNG 5. CÀI ĐẶT VÀ TRIỂN KHAI 22
5.1 Cấu trúc dự án 22
5.2 Môi trường và thư viện 22

5.3 Hướng dẫn cài đặt và chạy hệ thống 23
5.4 Tích hợp Swagger UI 24
5.5 Một số điểm kỹ thuật đáng chú ý 24
CHƯƠNG 6. KẾT QUẢ VÀ DEMO 25
6.1 Giao diện Dashboard 25
CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 29
7.1 Kết quả đạt được 29
7.2 Hạn chế hiện tại 29
7.3 Hướng phát triển 30
TÀI LIỆU THAM KHẢO 31

DANH MỤC CÁC HÌNH VẼ

Hình 1 Sơ đồ Use Case tổng quan hệ thống 9
Hình 2 Sơ đồ kiến trúc tổng quan hệ thống 14
Hình 3 Sơ đồ ERD hệ thống 21
Hình 4 Cấu trúc thư mục dự án 22
Hình 5 Giao diện Swagger UI tại /api-docs 24
Hình 6 Giao diện Dashboard 25
Hình 7 Giao diện Tab Data Sensor 26
Hình 8 Giao diện Tab Action History 27
Hình 9 Giao diện Tab Developer Profile 27
Hình 10 Trạng thái Loading khi điều khiển thiết bị 28
Hình 11 Toast Notification cảnh báo ánh sáng thấp 28

DANH MỤC CÁC BẢNG BIỂU

Bảng 1 Công nghệ sử dụng trong hệ thống 7
Bảng 2 Mô tả các tác nhân trong hệ thống 9
Bảng 3 Đặc tả Use Case: Điều khiển thiết bị 9
Bảng 4 Đặc tả Use Case: Xem dữ liệu cảm biến thời gian thực 10
Bảng 5 Đặc tả Use Case: Xem dữ liệu cảm biến lưu trong CSDL 11
Bảng 6 Đặc tả Use Case: Xem lịch sử điều khiển 11
Bảng 7 Đặc tả Use Case: Xem Profile 12
Bảng 8 Đặc tả Use Case: Đồng bộ trạng thái khi tải lại trang 12
Bảng 9 Mô tả các topic MQTT 15
Bảng 10 Mã trạng thái HTTP áp dụng trong hệ thống 16
Bảng 11 Mô tả bảng user 19
Bảng 12 Mô tả bảng device 19
Bảng 13 Mô tả bảng sensor 19
Bảng 14 Mô tả bảng sensor_data 20
Bảng 15 Mô tả bảng action_history 20
Bảng 16 Danh sách thư viện phụ thuộc 23

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Thuật ngữ tiếng Anh/Giải thích	Thuật ngữ tiếng Việt/Giải thích
LMS	Learning Management System	Hệ thống quản lý học tập

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong bối cảnh cách mạng công nghiệp 4.0 đang diễn ra mạnh mẽ, Internet of Things (IoT) đã trở thành một trong những nền tảng công nghệ cốt lõi định hình lại cách con người tương tác với môi trường sống xung quanh. Các hệ thống nhà thông minh (Smart Home) không còn là khái niệm xa lạ mà đang dần trở thành xu hướng phổ biến, mang lại khả năng giám sát, tự động hóa và điều khiển từ xa các thiết bị trong không gian sinh hoạt hàng ngày.

Tuy nhiên, một bài toán thực tiễn đặt ra là làm thế nào để xây dựng một hệ thống IoT hoàn chỉnh - từ tầng phần cứng vi điều khiển, tầng giao tiếp giao thức, tầng xử lý dữ liệu backend, đến tầng giao diện người dùng - với chi phí thấp, độ trễ tối thiểu và khả năng mở rộng linh hoạt. Đây chính là động lực thúc đẩy việc nghiên cứu và triển khai đề tài này.

1.2 Mục đích và phạm vi đề tài

Đề tài tập trung xây dựng một hệ thống IoT hoàn chỉnh phục vụ giám sát môi trường không gian trong nhà và điều khiển thiết bị từ xa theo thời gian thực. Hệ thống được thiết kế theo kiến trúc phân tầng rõ ràng, tích hợp đồng thời ba giao thức truyền thông gồm MQTT, WebSocket và RESTful HTTP nhằm đáp ứng các yêu cầu về tốc độ, độ tin cậy và tính tương thích.

Phạm vi của đề tài bao gồm:

- Thiết kế và lập trình vi điều khiển ESP8266 để đọc dữ liệu từ các cảm biến nhiệt độ, độ ẩm và ánh sáng, đồng thời điều khiển các relay.
- Xây dựng Backend Server bằng Node.js/Express.js tích hợp MQTT Client, Socket.io và RESTful API.
- Thiết kế cơ sở dữ liệu MySQL lưu trữ dữ liệu cảm biến và lịch sử thao tác.
- Phát triển giao diện Web Dashboard trực quan hiển thị dữ liệu thời gian thực và cho phép điều khiển thiết bị.
- Tích hợp tài liệu API chuẩn OpenAPI 3.0 thông qua Swagger UI.

1.3 Mục tiêu chính

Các mục tiêu cụ thể của đề tài bao gồm:

Giám sát môi trường thời gian thực: Thu thập và hiển thị liên tục các chỉ số nhiệt độ, độ ẩm và cường độ ánh sáng với chu kỳ cập nhật 2 giây, trực quan hóa bằng biểu đồ đường động trên giao diện web.

Điều khiển thiết bị hai chiều: Cung cấp khả năng bật/tắt các thiết bị relay từ xa thông qua giao diện web, đảm bảo cơ chế phản hồi trạng thái hai chiều (LOADING → SUCCESS/FAILED) để xác thực kết quả thực thi lệnh.

Tự động hóa cơ bản: Triển khai quy tắc tự động bật đèn khi cường độ ánh sáng môi trường xuống dưới ngưỡng cho phép (50 Lux), kèm cảnh báo trực quan tới người dùng.

Lưu trữ và tra cứu dữ liệu: Ghi nhận toàn bộ dữ liệu cảm biến và lịch sử thao tác vào cơ sở dữ liệu MySQL, hỗ trợ truy vấn phân trang, tìm kiếm và lọc đa tiêu chí.

Tài liệu hóa API: Cung cấp tài liệu API chuẩn OpenAPI 3.0 tích hợp trực tiếp trên server, hỗ trợ kiểm thử live các endpoint.

1.4 Công nghệ sử dụng

Bảng 1 Công nghệ sử dụng trong hệ thống

Tầng	**Công nghệ**	**Mô tả**
Phần cứng	ESP8266, C++/Arduino	Vi điều khiển đọc cảm biến và điều khiển relay
Giao thức IoT	MQTT – HiveMQ Broker	Giao tiếp nhẹ giữa phần cứng và server
Backend	Node.js, Express.js	Xử lý logic nghiệp vụ, REST API
Realtime	Socket.io (WebSocket)	Đẩy dữ liệu tức thì từ server xuống Web UI
Cơ sở dữ liệu	MySQL	Lưu trữ dữ liệu cảm biến và lịch sử thao tác
Frontend	HTML5, CSS3, JavaScript ES6+	Giao diện người dùng
Trực quan hóa	Chart.js	Biểu đồ đường dữ liệu cảm biến
Tài liệu API	Swagger UI, OpenAPI 3.0	Mô tả và kiểm thử API

CHƯƠNG 2. PHÂN TÍCH YÊU CẦU HỆ THỐNG

2.1 Yêu cầu chức năng

Hệ thống cần đáp ứng sáu nhóm chức năng chính sau:

Chức năng 1 – Giám sát môi trường thời gian thực: Hệ thống liên tục thu nhận dữ liệu từ các cảm biến và hiển thị tức thì lên giao diện Web mà không yêu cầu người dùng tải lại trang. Dữ liệu bao gồm nhiệt độ (°C), độ ẩm (%) và cường độ ánh sáng (Lux), được cập nhật mỗi 2 giây qua kênh WebSocket.

Chức năng 2 – Điều khiển thiết bị từ xa: Người dùng có thể bật/tắt từng relay (Quạt, Đèn) thông qua công tắc toggle trên Dashboard. Hệ thống thực thi lệnh qua MQTT và phản hồi kết quả thực thi (SUCCESS/FAILED) về giao diện theo cơ chế xác nhận hai chiều.

Chức năng 3 – Cảnh báo và tự động hóa: Khi cường độ ánh sáng phòng xuống dưới 50 Lux, hệ thống tự động phát cảnh báo Toast Notification trên giao diện và kích hoạt lệnh bật relay đèn nếu đèn đang ở trạng thái tắt.

Chức năng 4 – Tra cứu lịch sử dữ liệu cảm biến: Cung cấp bảng dữ liệu phân trang cho phép người dùng xem lại toàn bộ lịch sử đo lường của từng cảm biến, hỗ trợ tìm kiếm theo giá trị hoặc thời gian và lọc theo loại cảm biến.

Chức năng 5 – Tra cứu lịch sử điều khiển: Lưu trữ và hiển thị toàn bộ nhật ký thao tác điều khiển thiết bị bao gồm tên thiết bị, loại hành động, trạng thái thực thi và thời điểm thực hiện. Hỗ trợ lọc theo thiết bị, loại hành động và trạng thái.

Chức năng 6 – Đồng bộ trạng thái khi tải lại trang: Khi người dùng tải lại trình duyệt (F5), giao diện tự động gọi API lấy trạng thái hiện tại của tất cả thiết bị và hiển thị đúng vị trí ON/OFF của từng công tắc relay.

2.2 Yêu cầu phi chức năng

Hiệu năng: Độ trễ từ khi cảm biến đọc dữ liệu đến khi hiển thị trên Dashboard không vượt quá 500ms trong điều kiện mạng ổn định.

Độ tin cậy: Cơ chế timeout 3.5 giây đảm bảo giao diện không bị treo vô hạn khi phần cứng không phản hồi; hệ thống tự động revert trạng thái công tắc về vị trí cũ và ghi nhận lỗi FAILED.

Bảo mật cơ bản: Mật khẩu người dùng được lưu dưới dạng hash trong cơ sở dữ liệu, thông tin cấu hình nhạy cảm (mật khẩu DB, thông tin MQTT) cần được tách ra file môi trường riêng.

Khả năng mở rộng: Kiến trúc phân tầng rõ ràng (routes → controllers → services) cho phép dễ dàng bổ sung thêm thiết bị, cảm biến hoặc tính năng mới mà không ảnh hưởng đến các module hiện có.

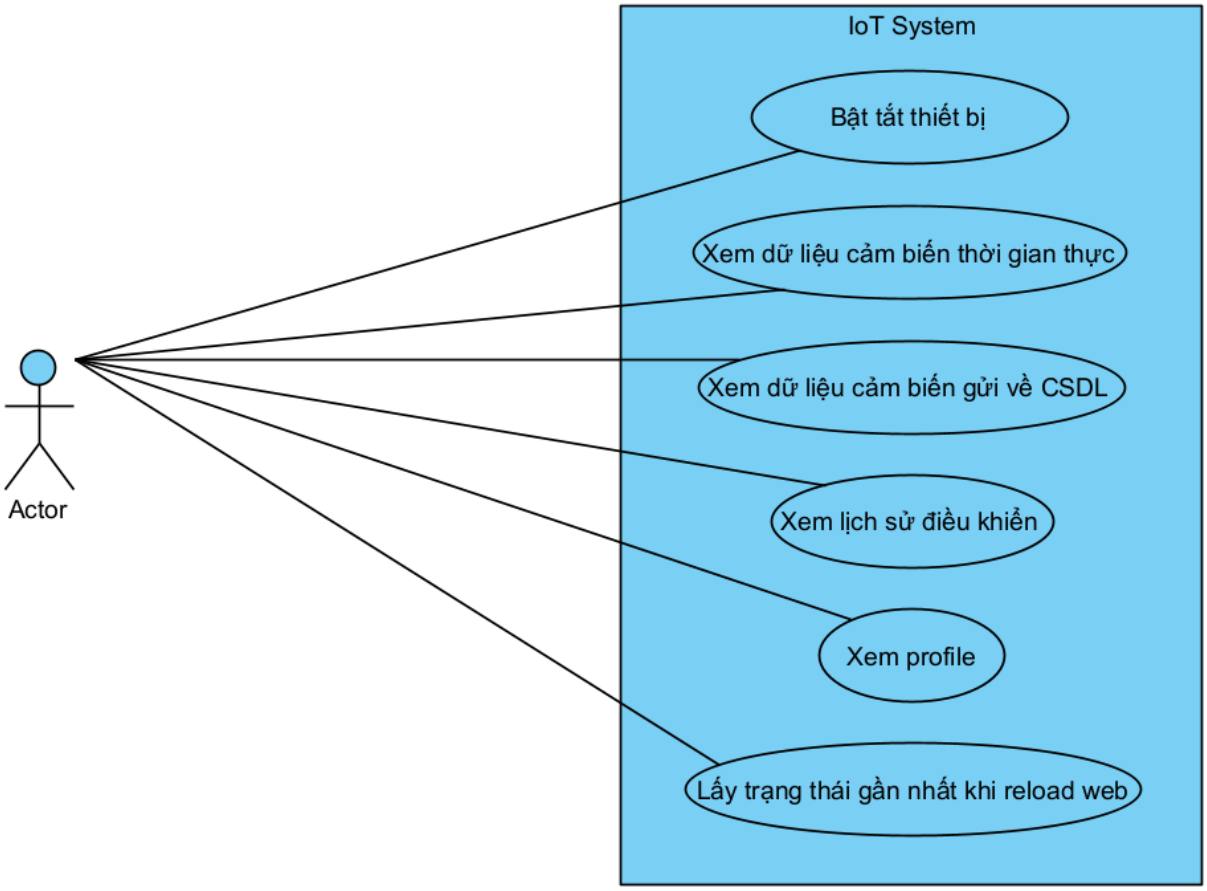
Tính khả dụng: Giao diện Web hiển thị tốt trên các trình duyệt hiện đại, dữ liệu được cập nhật liên tục trong suốt phiên làm việc mà không cần can thiệp của người dùng.

2.3 Các tác nhân trong hệ thống

Bảng 2 Mô tả các tác nhân trong hệ thống

Tác nhân	**Loại**	**Mô tả**
User (Người dùng)	Tác nhân chính	Người truy cập Web Dashboard để giám sát và điều khiển thiết bị
ESP8266	Tác nhân phần cứng	Vi điều khiển thu thập dữ liệu cảm biến và thực thi lệnh điều khiển relay
MQTT Broker (HiveMQ)	Tác nhân hệ thống	Trung gian truyền thông điệp giữa ESP8266 và Backend Server
Backend Server	Tác nhân hệ thống	Xử lý logic nghiệp vụ, lưu trữ CSDL, trung chuyển dữ liệu

2.4 Sơ đồ Use Case tổng quan



Hình 1 Sơ đồ Use Case tổng quan hệ thống

2.5 Đặc tả chi tiết các Use Case

Bảng 3 Đặc tả Use Case: Điều khiển thiết bị

Tên usecase	Điều khiển thiết bị
Actor	User

Mô tả	Cho phép người dùng điều khiển đóng/ngắt các thiết bị ngoại vi từ xa thông qua giao diện Web. Hệ thống ghi nhận mọi biến động trạng thái dưới dạng các bản ghi nối tiếp (Append-Only Log).
Tiền điều kiện	Người dùng đang truy cập Dashboard. Thiết bị ESP32 được cấp nguồn và duy trì kết nối mạng với MQTT Broker.
Hậu điều kiện	Thiết bị phần cứng thay đổi trạng thái, 2 bản ghi lịch sử nối tiếp (LOADING và SUCCESS/FAILED) được thêm mới vào cơ sở dữ liệu.
Kịch bản chính	1. User click vào công tắc (ON/OFF) của thiết bị trên WebUI. 2. Giao diện chuyển nút sang trạng thái LOADING (khóa nút bấm) và gửi POST request lên Backend. 3. Backend thực hiện INSERT bản ghi thứ nhất vào bảng ActionHistory với trạng thái LOADING. 4. Backend publish lệnh điều khiển (JSON) vào topic device_control qua MQTT Broker. 5. ESP32 nhận lệnh và kích hoạt Relay phần cứng. 6. ESP32 publish phản hồi kết quả vào topic device_response. 7. Backend nhận xác nhận, thực hiện INSERT bản ghi thứ hai vào bảng ActionHistory với trạng thái SUCCESS/FAILED. 8. Backend trả HTTP 200 về WebUI để mở khóa nút và gạt công tắc sang vị trí mới.
Ngoại lệ	- Quá hạn (Timeout): Quá 5s không có phản hồi từ ESP32, Backend lưu trạng thái TIMEOUT, UI khôi phục nút về cũ và báo lỗi. - Lỗi phần cứng: ESP32 trả về lỗi, Backend lưu trạng thái FAILED, UI báo lỗi.

Bảng 4 Đặc tả Use Case: Xem dữ liệu cảm biến thời gian thực

Tên usecase	Xem dữ liệu cảm biến thời gian thực
Actor	User
Mô tả	Người dùng theo dõi sự thay đổi của các chỉ số môi trường (Nhiệt độ, Độ ẩm, Ánh sáng) liên tục trên Dashboard.
Tiền điều kiện	Người dùng đang truy cập Dashboard, các cảm biến hoạt động bình thường, kết nối MQTT và WebSocket ổn định.
Hậu điều kiện	Dữ liệu mới nhất được lưu vào CSDL và cập nhật tức thời lên biểu đồ, thẻ thông tin trên UI.
Kịch bản chính	1. ESP32 đọc dữ liệu từ cảm biến định kỳ mỗi 2 giây.

	2. ESP32 đóng gói payload JSON và Publish lên topic data_sensor. 3. Backend (đã Subscribe) tiếp nhận gói tin, bóc tách và INSERT vào bảng SensorData. 4. Cùng lúc, Backend đẩy dữ liệu qua WebSocket sang WebUI. 5. WebUI nhận luồng dữ liệu, cập nhật thẻ số và vẽ tiếp điểm mới trên Line Chart không cần tải lại trang.
Ngoại lệ	- Mất kết nối mạng, luồng WebSocket ngắt, dữ liệu trên màn hình sẽ đứng yên ở mốc cuối cùng nhận được.

Bảng 5 Đặc tả Use Case: Xem dữ liệu cảm biến lưu trong CSDL

Tên usecase	Xem dữ liệu cảm biến gửi về CSDL
Actor	User
Mô tả	Truy xuất, rà soát lại toàn bộ lịch sử các mốc đo lường cảm biến đã được lưu trữ trong quá khứ.
Tiền điều kiện	Truy cập thành công vào tab "Data Sensor" trên thanh điều hướng.
Hậu điều kiện	Danh sách dữ liệu hiển thị thành bảng chi tiết, có phân trang.
Kịch bản chính	6. User click điều hướng sang trang Data Sensor. 7. WebUI gửi API request lấy dữ liệu (bao gồm các biến phân trang page, limit). 8. Backend kết nối CSDL, truy vấn bảng SensorData. 9. Backend trả về khối dữ liệu JSON cho WebUI. 10. WebUI render dữ liệu ra bảng, hiển thị theo từng loại cảm biến và thời gian.
Ngoại lệ	- Không có dữ liệu trong Database, hệ thống trả về mảng rỗng và hiển thị bảng trống (No data).

Bảng 6 Đặc tả Use Case: Xem lịch sử điều khiển

Tên usecase	Xem lịch sử điều khiển
Actor	User
Mô tả	Người dùng kiểm tra lại nhật ký các phiên thao tác điều khiển thiết bị (ai thao tác, thiết bị nào, thời gian, kết quả).

Tiền điều kiện	Truy cập thành công vào tab "Action History" trên thanh điều hướng.
Hậu điều kiện	Bảng lịch sử điều khiển được tải lên giao diện với định dạng màu sắc phân biệt theo trạng thái (SUCCESS, FAILED, TIMEOUT).
Kịch bản chính	11. User truy cập vào trang Action History. 12. WebUI gọi API request lấy nhật ký điều khiển, kèm tham số lọc (nếu có). 13. Backend truy vấn bảng ActionHistory (kết hợp bảng Device để lấy tên). 14. Trả mảng dữ liệu JSON về cho Frontend. 15. Frontend render bảng, hiển thị đầy đủ thông tin Log ID, tên thiết bị, hành động và trạng thái thi hành.
Ngoại lệ	- Lỗi kết nối CSDL, trả về HTTP 500, WebUI hiển thị popup thông báo gián đoạn hệ thống.

Bảng 7 Đặc tả Use Case: Xem Profile

Tên usecase	Xem profile
Actor	User
Mô tả	Xem thông tin định danh của nhà phát triển, tài khoản quản trị và chi tiết môi trường học thuật.
Tiền điều kiện	Truy cập thành công vào tab "Profile" trên thanh điều hướng.
Hậu điều kiện	Hiển thị thẻ ID card và các thông tin liên kết tĩnh.
Kịch bản chính	1. User click vào menu Profile. 2. WebUI gọi API (hoặc load dữ liệu cục bộ) để lấy thông tin. 3. Hệ thống trả về thông tin cá nhân (họ và tên, mã sinh viên, link github,..). 4. WebUI render khối giao diện Academic Details và Developer Profile.
Ngoại lệ	- Xảy ra độ trễ tải trang, giao diện hiển thị các khối xám (Skeleton Loading) chờ đến khi có dữ liệu.

Bảng 8 Đặc tả Use Case: Đồng bộ trạng thái khi tải lại trang

Tên usecase	Lấy trạng thái gần nhất khi reload web
Actor	User

Mô tả	Tự động đồng bộ toàn bộ giao diện (trạng thái công tắc ON/OFF, chỉ số cảm biến) khớp với thực tế thiết bị ngay khi user F5 tải lại trang.
Tiền điều kiện	User tải lại trang trình duyệt (Reload/F5).
Hậu điều kiện	Giao diện hiển thị chính xác thiết bị nào đang bật/tắt ở thời điểm hiện tại.
Kịch bản chính	5. Trình duyệt tải lại DOM và vòng đời của WebUI khởi động. 6. Quá trình mounted gọi ngay API GET /telemetry/latest. 7. Backend truy xuất bản ghi cảm biến mới nhất và trạng thái cuối cùng của thiết bị trong DB. 8. Trả dữ liệu khởi tạo về. 9. WebUI gán giá trị này để hiển thị màu sắc/vị trí của công tắc Relay.
Ngoại lệ	- API lỗi hoặc mạng rớt ngay lúc tải, WebUI gán giá trị mặc định (N/A, OFF) và chờ tín hiệu WebSocket tiếp theo.

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

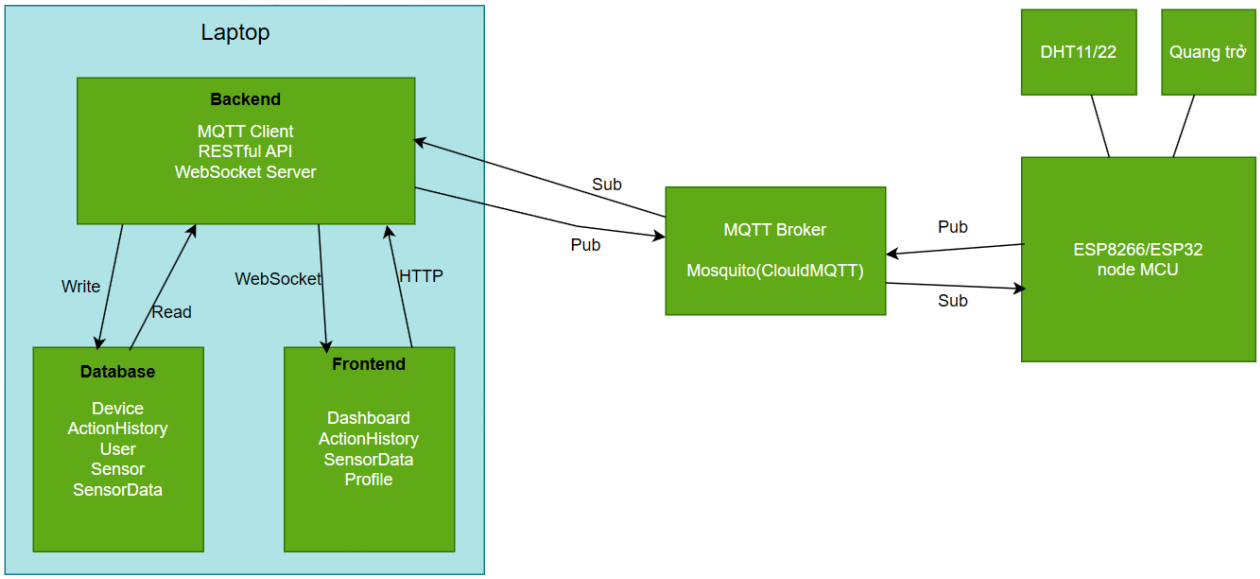
3.1 Kiến trúc tổng quan

Hệ thống được tổ chức theo mô hình kiến trúc ba tầng kết hợp giao thức Publish/Subscribe:

Tầng Phần cứng (Hardware Layer): Vi điều khiển ESP8266 đóng vai trò là đầu cuối IoT, chịu trách nhiệm đọc dữ liệu từ cảm biến DHT11 (nhiệt độ, độ ẩm) và cảm biến quang (ánh sáng), đồng thời điều khiển các module relay theo lệnh nhận về. ESP8266 giao tiếp với tầng trung gian thông qua giao thức MQTT.

Tầng Trung gian – Backend Server (Application Layer): Node.js/Express.js Server đóng vai trò trung tâm xử lý, đảm nhiệm đồng thời ba vai trò: MQTT Client (Subscribe/Publish với ESP8266), Socket.io Server (đẩy dữ liệu realtime xuống Web UI) và RESTful API Server (phục vụ truy vấn dữ liệu từ Frontend). Backend kết nối với cơ sở dữ liệu MySQL để lưu trữ và truy xuất dữ liệu.

Tầng Giao diện (Presentation Layer): Web Dashboard xây dựng bằng HTML5/CSS3/JavaScript thuần, nhận dữ liệu realtime qua WebSocket và gửi lệnh điều khiển lên Backend. Giao diện bao gồm bốn tab chức năng: Dashboard, Data Sensor, Action History và Profile.



Hình 2 Sơ đồ kiến trúc tổng quan hệ thống

3.2 Luồng dữ liệu

3.2.1 Luồng thu thập và hiển thị dữ liệu cảm biến

[Hình 2.2: Luồng thu thập và hiển thị dữ liệu cảm biến]

Định kỳ mỗi 2 giây, ESP8266 đọc các thông số từ cảm biến và đóng gói thành payload JSON theo định dạng:

```
{
  "temperature": 28.5,
```

"humidity": 65.0,
"light": 450
}

ESP8266 Publish payload này lên topic vanh_iot/data_sensor trên HiveMQ Broker. Backend Server (đã Subscribe topic này) tiếp nhận gói tin, bóc tách dữ liệu và thực hiện đồng thời hai tác vụ: INSERT ba bản ghi vào bảng sensor_data trong MySQL và emit sự kiện sensor_telemetry qua Socket.io tới toàn bộ các client đang kết nối. Web UI lắng nghe sự kiện này và cập nhật ngay lập tức các thẻ hiển thị số liệu cũng như vẽ thêm điểm mới trên biểu đồ đường.

3.2.2 Luồng điều khiển thiết bị hai chiều

Khi người dùng tác động vào công tắc toggle trên Dashboard, Web UI lập tức khóa nút bấm, chuyển sang trạng thái Loading (hiệu ứng spinner) và emit sự kiện send_control qua Socket.io lên Backend kèm payload { deviceId, action }. Backend xử lý theo các bước sau:

1. Ghi bản ghi đầu tiên vào action_history với status = 'LOADING' và emit action_logged về UI.
2. Publish lệnh điều khiển JSON vào topic vanh_iot/device_control.
3. Đặt bộ đếm timeout 3.5 giây và lưu thông tin pending vào Map.
4. ESP8266 nhận lệnh, kích hoạt relay tương ứng và Publish phản hồi vào topic vanh_iot/device_response.
5. Backend nhận phản hồi, xóa pending khỏi Map, UPDATE trạng thái trong bảng device, ghi bản ghi SUCCESS vào action_history và emit device_state_sync về UI để mở khóa và cập nhật giao diện.

Nếu hết timeout mà không nhận được phản hồi, Backend ghi bản ghi FAILED, revert trạng thái thiết bị về giá trị cũ và thông báo lỗi về giao diện.

3.3 Thiết kế giao thức MQTT

Bảng 9 Mô tả các topic MQTT

Topic	**Publisher**	**Subscriber**	**Mô tả**
vanh_iot/data_sensor	ESP8266	Backend	Dữ liệu cảm biến định kỳ 2s
vanh_iot/device_control	Backend	ESP8266	Lệnh điều khiển relay
vanh_iot/device_response	ESP8266	Backend	Xác nhận kết quả thực thi lệnh
vanh_iot/device_sync	ESP8266	Backend	Yêu cầu đồng bộ trạng thái khi ESP8266 khởi động

3.4 Thiết kế RESTful API

Hệ thống cung cấp các endpoint RESTful phục vụ truy vấn dữ liệu từ Frontend. Base URL: http://localhost:5000/api-docs.

Bảng 10 Mã trạng thái HTTP áp dụng trong hệ thống

Mã	**Phân loại**	**Diễn giải**
200 OK	Thành công	Yêu cầu được thực thi trọn vẹn
400 Bad Request	Lỗi yêu cầu	Dữ liệu gửi lên sai định dạng
404 Not Found	Không tìm thấy	Tài nguyên yêu cầu không tồn tại
500 Internal Error	Lỗi máy chủ	Lỗi xử lý nội bộ hoặc CSDL

3.4.1 API Dữ liệu cảm biến

GET /api/sensors – Lấy danh sách dữ liệu cảm biến có phân trang và tìm kiếm.

Query Parameters:

- page (int, default: 1): Số trang hiện tại
- limit (int, default: 20): Số bản ghi mỗi trang
- field (string): Trường cần tìm kiếm (Time / Temperature / Humidity / Ambient Light)
- keyword (string): Từ khóa tìm kiếm

Response mẫu:


```
{
 "total": 1250,
 "page": 1,
 "limit": 20,
 "data": [
  {
   "data_id": 1024,
   "sensor_type": "Temperature",
   "unit": "°C",
   "reading_value": 28.5,
   "recorded_at": "2026-10-21 14:32:07"
  }
 ]
}
```

3.4.2 API Lịch sử điều khiển

GET /api/actions – Lấy nhật ký thao tác điều khiển thiết bị có phân trang và lọc.

Query Parameters:

- page, limit: Phân trang
- device (string): Lọc theo tên thiết bị
- action (string): Lọc theo hành động (All / ON / OFF)
- status (string): Lọc theo trạng thái (All / SUCCESS / FAILED / LOADING)
- keyword (string): Tìm kiếm theo thời gian

Response mẫu:


```
{
 "total": 2847,
 "page": 1,
 "limit": 20,
 "data": [
  {
   "action_id": 501,
   "device_name": "Relay Fan 01",
   "user_name": "Trần Việt Anh",
   "action": "ON",
   "status": "SUCCESS",
   "executed_at": "2026-10-21 14:32:07"
  },
  {
   "action_id": 500,
   "device_name": "Relay Light 01",
   "user_name": "Trần Việt Anh",
   "action": "OFF",
   "status": "FAILED",
   "executed_at": "2026-10-21 14:30:15"
  }
 ]
```

```
}
```


3.4.3 API Thiết bị

GET /api/devices – Lấy danh sách toàn bộ thiết bị và trạng thái hiện tại.

Query Parameters: Không yêu cầu

Response mẫu:


```
[
  {
    "device_id": 1,
    "device_name": "Relay Fan 01",
    "pin": 14,
    "status": 1,
    "last_connected": "2026-10-21 14:30:00"
  }
]
```


3.5 Thiết kế giao diện người dùng

Giao diện Web Dashboard được thiết kế theo phong cách Dark Theme hiện đại, lấy cảm hứng từ các công cụ monitoring chuyên nghiệp. Hệ thống màu sắc sử dụng nền xanh đen đậm kết hợp với các tông xanh dương, xanh lá, đỏ và vàng để phân biệt trạng thái và loại dữ liệu.

Giao diện được chia thành bốn tab chức năng điều hướng qua Sidebar:

- Tab Dashboard: Hiển thị ba thẻ số liệu cảm biến (nhiệt độ, độ ẩm, ánh sáng), biểu đồ đường thời gian thực và hai công tắc điều khiển relay.
- Tab Data Sensor: Bảng dữ liệu lịch sử cảm biến với chức năng tìm kiếm theo lựa chọn và phân trang tùy chỉnh.
- Tab Action History: Bảng nhật ký điều khiển thiết bị với bộ lọc đa tiêu chí và màu sắc badge phân biệt trạng thái.
- Tab Profile: Thẻ thông tin sinh viên, chi tiết học thuật và các liên kết dự án.

CHƯƠNG 4. CƠ SỞ DỮ LIỆU

4.1 Mô tả các thực thể

Cơ sở dữ liệu iot_lab bao gồm năm bảng chính với vai trò và cấu trúc như sau:

Bảng 11 Mô tả bảng user

Cột	**Kiểu dữ liệu**	**Ràng buộc**	**Mô tả**
user_id	INT(10)	PK, AUTO_INCREMENT	Định danh người dùng
user_name	VARCHAR(50)	NOT NULL	Tên hiển thị
password_hash	VARCHAR(255)	NOT NULL	Mật khẩu đã hash
email	VARCHAR(100)	NOT NULL	Địa chỉ email
created_at	TIMESTAMP	DEFAULT NOW()	Thời điểm tạo tài khoản

Bảng 12 Mô tả bảng device

Cột	**Kiểu dữ liệu**	**Ràng buộc**	**Mô tả**
device_id	INT(10)	PK, AUTO_INCREMENT	Định danh thiết bị
device_name	VARCHAR(50)	NULL	Tên thiết bị
pin	INT(10)	NOT NULL	Số chân GPIO điều khiển
status	TINYINT(1)	DEFAULT 0	Trạng thái hiện tại (0=OFF, 1=ON)
last_connected	TIMESTAMP	ON UPDATE NOW()	Lần cập nhật trạng thái gần nhất

Bảng 13 Mô tả bảng sensor

Cột	**Kiểu dữ liệu**	**Ràng buộc**	**Mô tả**
sensor_id	INT(10)	PK, AUTO_INCREMENT	Định danh cảm biến
sensor_type	VARCHAR(50)	NOT NULL	Loại cảm biến
unit	VARCHAR(50)	NULL	Đơn vị đo lường

Bảng 14 Mô tả bảng sensor_data

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
data_id	BIGINT(19)	PK, AUTO_INCREMENT	Định danh bản ghi đo lường
sensor_id	INT(10)	FK → sensor	Tham chiếu loại cảm biến
reading_value	FLOAT	NOT NULL	Giá trị đo được
recorded_at	TIMESTAMP	DEFAULT NOW()	Thời điểm ghi nhận

Bảng 15 Mô tả bảng action_history

Cột	Kiểu dữ liệu	Ràng buộc	Mô tả
action_id	INT(10)	PK, AUTO_INCREMENT	Định danh bản ghi lịch sử
device_id	INT(10)	FK → device	Thiết bị được điều khiển
user_id	INT(10)	FK → user	Người thực hiện thao tác
action	VARCHAR(50)	NOT NULL	Hành động (ON/OFF)
executed_at	TIMESTAMP	DEFAULT NOW()	Thời điểm thực hiện
status	VARCHAR(10)	NOT NULL	Kết quả (SUCCESS/LOADING/FAILED)

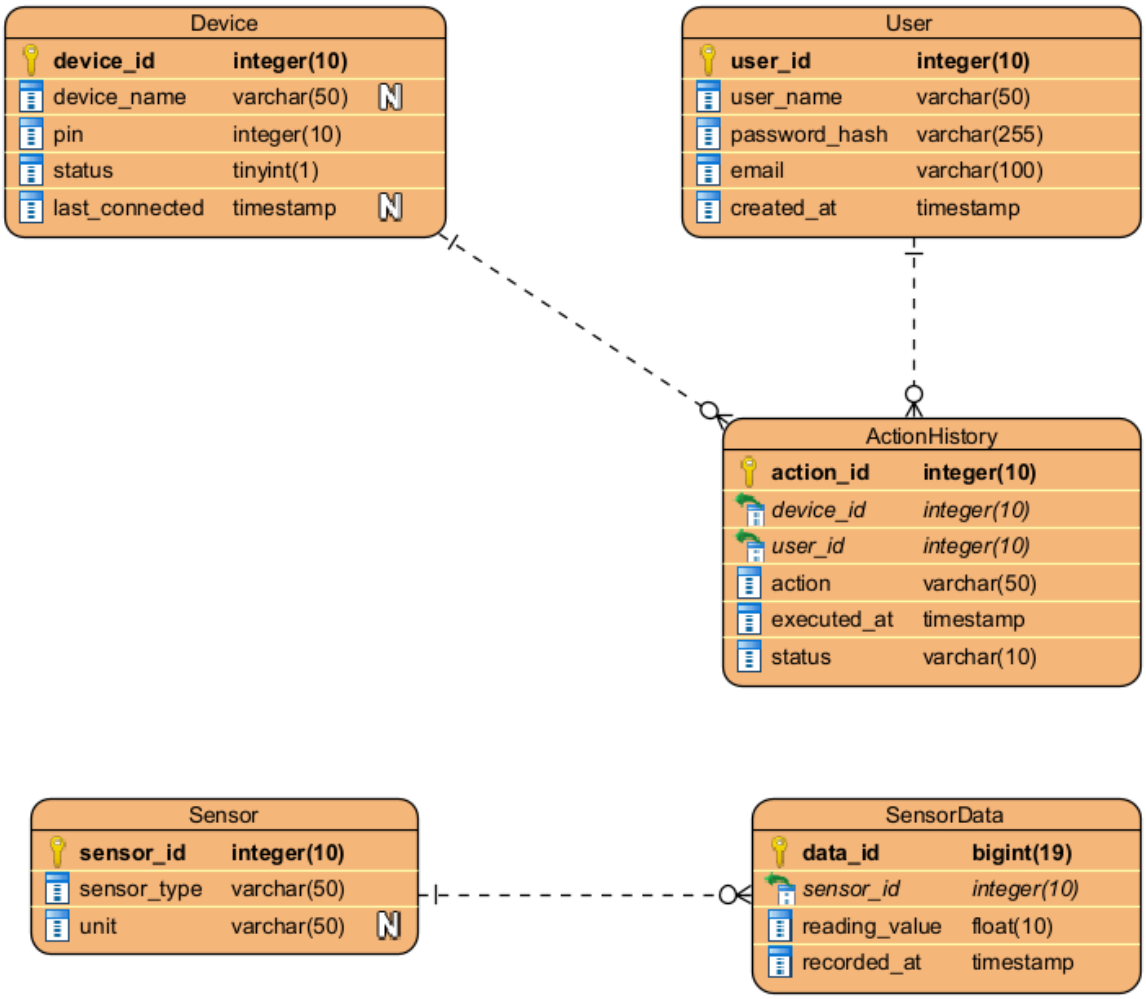
4.2 Mô tả các quan hệ

Quan hệ User – ActionHistory (1:N): Một người dùng có thể thực hiện nhiều thao tác điều khiển thiết bị. Mỗi bản ghi trong action_history bắt buộc tham chiếu đến một người dùng cụ thể qua khóa ngoại user_id.

Quan hệ Device – ActionHistory (1:N): Một thiết bị có thể tiếp nhận nhiều lượt điều khiển qua các phiên làm việc. Mỗi bản ghi lịch sử tham chiếu thiết bị tương ứng qua khóa ngoại device_id.

Quan hệ Sensor – SensorData (1:N): Mỗi cảm biến liên tục tạo ra nhiều bản ghi đo lường theo chu kỳ. Các bản ghi này tham chiếu cảm biến tương ứng qua khóa ngoại sensor_id.

4.3 Sơ đồ thực thể - quan hệ (ERD)



Hình 3 Sơ đồ ERD hệ thống

4.3 Dữ liệu khởi tạo

Hệ thống được khởi tạo với dữ liệu mặc định khi thiết lập lần đầu:

- Người dùng quản trị: Tài khoản B23DCAT021 – Trần Việt Anh với user_id = 1.
- Hai thiết bị relay: Relay Fan 01 và Relay Light 01.
- Ba loại cảm biến: Temperature (°C), Humidity (%), Ambient Light (Lux).

CHƯƠNG 5. CÀI ĐẶT VÀ TRIỂN KHAI

5.1 Cấu trúc dự án

Kiến trúc mã nguồn Backend tuân theo mô hình phân tầng rõ ràng: Routes chỉ định tuyến request, Controllers xử lý logic nghiệp vụ, Services thực hiện các tác vụ phức tạp dùng chung (như deviceService.js xử lý toàn bộ luồng điều khiển thiết bị với timeout/fallback), Config tập trung cấu hình kết nối.

```
IoT/
├── config/              # Cấu hình kết nối MySQL và MQTT
│   ├── db.js            # MySQL connection pool
│   └── mqtt.js          # MQTT client và topic constants
├── controllers/         # Xử lý logic nghiệp vụ API
│   ├── sensorController.js
│   ├── actionController.js
│   └── deviceController.js
├── mqtt/                # MQTT event handler
│   └── mqttHandler.js
├── routes/              # Định tuyến API
│   ├── sensorRoutes.js
│   ├── actionRoutes.js
│   └── deviceRoutes.js
├── services/            # Tầng tương tác nghiệp vụ
│   └── deviceService.js
├── sockets/             # Socket.io event handler
│   └── socketHandler.js
├── utils/               # Tiện ích dùng chung
│   └── timeUtil.js
├── public/              # Giao diện người dùng tĩnh
│   ├── index.html
│   ├── style.css
│   ├── dashboard.js
│   ├── sensor.js
│   ├── history.js
│   └── app.js
├── swagger.json         # Đặc tả OpenAPI 3.0
├── server.js            # Entry point
└── package.json
```

Hình 4 Cấu trúc thư mục dự án

5.2 Môi trường và thư viện

Bảng 16 Danh sách thư viện phụ thuộc

Thư viện	Phiên bản	Mục đích
express	5.2.1	Web framework xây dựng RESTful API
socket.io	4.8.3	WebSocket server cho realtime communication
mqtt	5.15.2	MQTT client kết nối HiveMQ Broker
mysql2	3.24.2	MySQL driver với hỗ trợ Promise/async-await
swagger-ui-express	5.0.1	Render Swagger UI từ file swagger.json
cors	2.8.6	Xử lý Cross-Origin Resource Sharing

5.3 Hướng dẫn cài đặt và chạy hệ thống

Bước 1 – Clone repository:

git clone https://github.com/vietanh1955/IoT.git

cd IoT

Bước 2 – Cài đặt thư viện phụ thuộc:

npm install

Bước 3 – Thiết lập cơ sở dữ liệu:

Mở MySQL client và thực thi lệnh tạo cơ sở dữ liệu:

CREATE DATABASE iot_lab CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

Sau đó import file database.sql để tạo các bảng và dữ liệu khởi tạo:

mysql -u root -p iot_lab < database.sql

Bước 4 – Cấu hình kết nối CSDL:

Mở file config/db.js và cập nhật thông số kết nối phù hợp với môi trường:

```
const pool = mysql.createPool({
host: 'localhost',
user: 'root',
password: 'your_password',
database: 'iot_lab'
});
```

Bước 5 – Khởi chạy server:

node server.js

Sau khi khởi động thành công, terminal hiển thị:

[HTTP Server] Running at: http://localhost:5000

[MQTT] Connected successfully to HiveMQ Broker

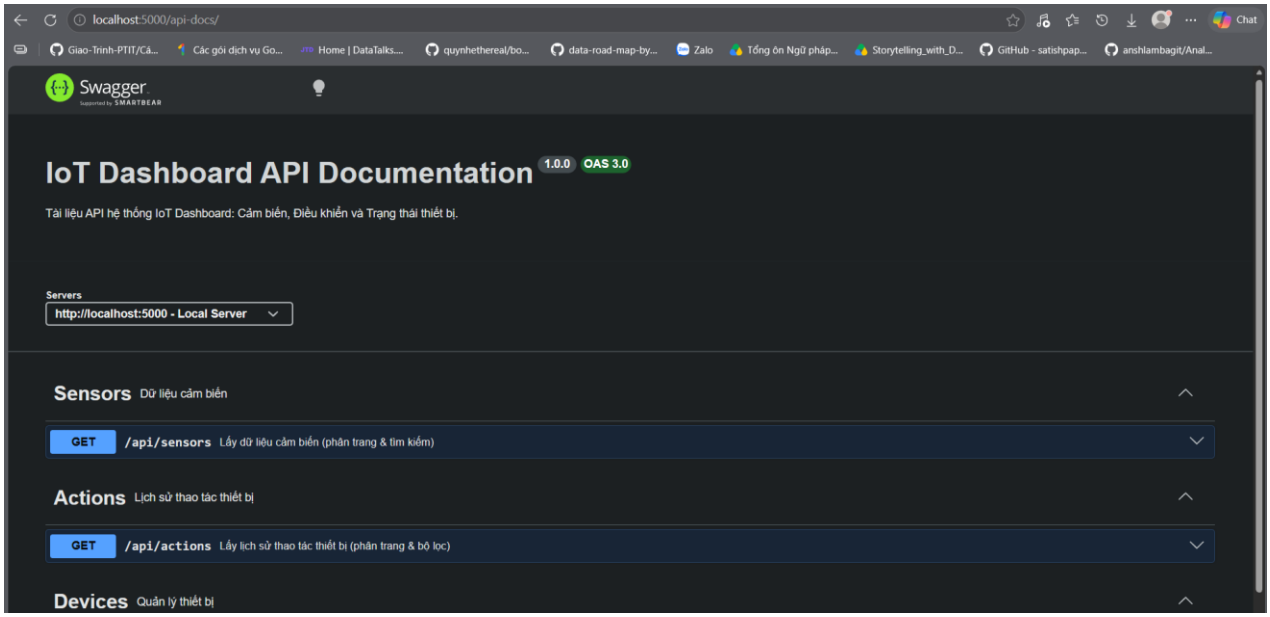
Bước 6 – Truy cập ứng dụng:

Dashboard: http://localhost:5000

API Documentation: http://localhost:5000/api-docs

5.4 Tích hợp Swagger UI

Tài liệu API được tích hợp trực tiếp trên server thông qua thư viện swagger-ui-express, đọc đặc tả từ file swagger.json độc lập theo chuẩn OpenAPI 3.0. Điểm mạnh của cách tiếp cận này là mã nguồn routes hoàn toàn sạch khỏi các annotation tài liệu, đồng thời Swagger UI cho phép kiểm thử trực tiếp từng endpoint với cơ sở dữ liệu thực qua tính năng "Try it out".



Hình 5 Giao diện Swagger UI tại /api-docs

5.5 Một số điểm kỹ thuật đáng chú ý

Xử lý múi giờ Việt Nam: Toàn bộ dữ liệu thời gian trong hệ thống được chuẩn hóa về múi giờ Asia/Ho_Chi_Minh thông qua hàm getVNTimeString() trong utils/timeUtil.js. Điều này đảm bảo tính nhất quán giữa dữ liệu lưu trong MySQL và thời gian hiển thị trên giao diện.

Cơ chế đồng bộ trạng thái khi ESP8266 khởi động: Khi ESP8266 kết nối lại sau khi mất điện hoặc restart, thiết bị Publish một gói tin vào topic vanh_iot/device_sync. Backend nhận tín hiệu này và lập tức gửi lại trạng thái hiện tại của toàn bộ thiết bị trong DB xuống cho ESP8266, đảm bảo phần cứng luôn đồng bộ với trạng thái logic trên server.

Pending Actions Map: Module deviceService.js sử dụng một Map trong bộ nhớ để theo dõi các lệnh điều khiển đang chờ phản hồi. Key của Map có dạng {deviceId}_{action} (ví dụ: 2_ON), ngăn chặn việc gửi lệnh trùng lặp khi người dùng bấm nhanh nhiều lần.

CHƯƠNG 6. KẾT QUẢ VÀ DEMO

6.1 Giao diện Dashboard

Tab Dashboard là giao diện chính của hệ thống, bao gồm ba thành phần chính:

Thẻ số liệu cảm biến (Sensor Cards): Ba thẻ hiển thị giá trị tức thời của nhiệt độ, độ ẩm và cường độ ánh sáng. Mỗi thẻ được cập nhật tự động mỗi 2 giây qua WebSocket mà không cần tải lại trang. Khi cường độ ánh sáng xuống dưới 50 Lux, thẻ Ambient Light chuyển sang viền đỏ với hiệu ứng nhấp nháy cảnh báo.

Biểu đồ đường thời gian thực (24-Hour Analytics): Biểu đồ đường sử dụng Chart.js hiển thị ba dataset song song: nhiệt độ (đỏ), độ ẩm (xanh dương) và ánh sáng (vàng). Trục Y bên trái dùng cho nhiệt độ và độ ẩm (0–100), trục Y bên phải dành riêng cho ánh sáng (0–1000 Lux). Biểu đồ giữ tối đa 8 điểm dữ liệu gần nhất và trượt dần theo thời gian thực.

Relay Controllers: Hai công tắc toggle điều khiển Relay Fan 01 và Relay Light 01. Khi kích hoạt, công tắc chuyển sang trạng thái Loading với hiệu ứng spinner xoay và bị khóa cho đến khi nhận được phản hồi từ phần cứng.



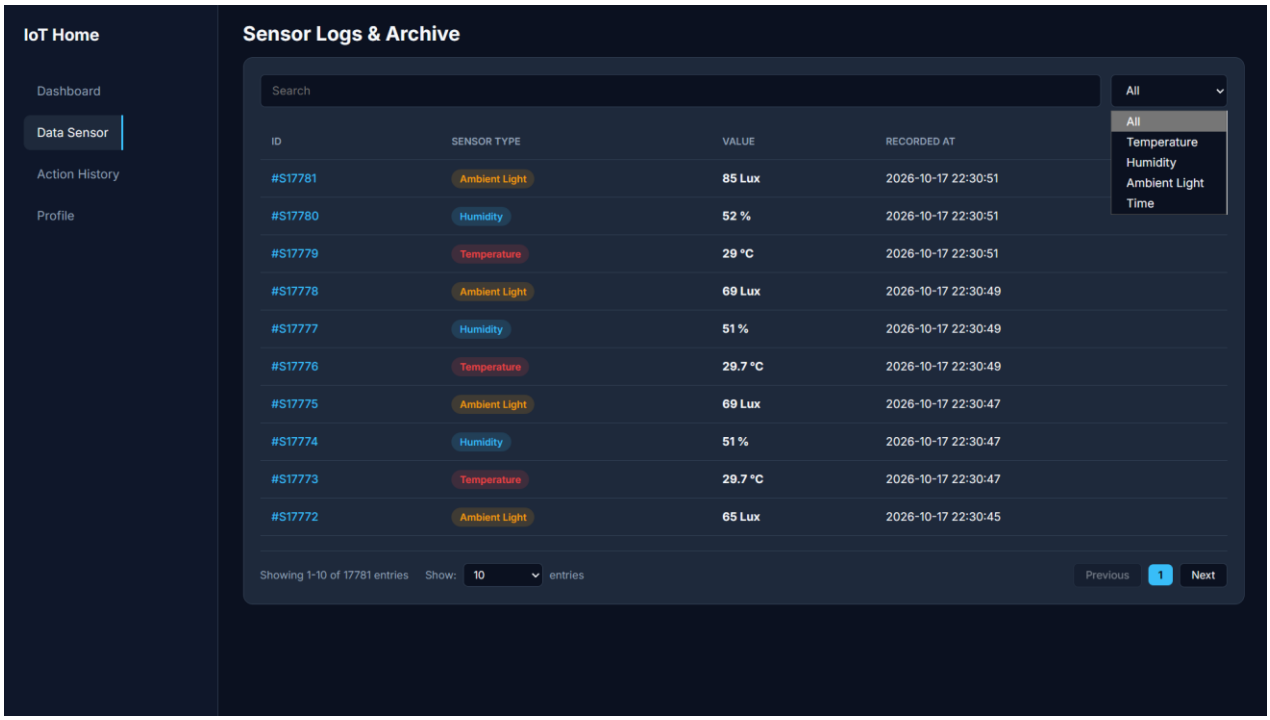
Hình 6 Giao diện Dashboard

6.2 Giao diện Tab Data Sensor

Tab Data Sensor hiển thị bảng lịch sử toàn bộ dữ liệu cảm biến đã lưu trong CSDL. Người dùng có thể:

- Tìm kiếm theo giá trị hoặc thời gian thông qua ô tìm kiếm.
- Lọc theo loại cảm biến (Temperature / Humidity / Ambient Light / Time).
- Tùy chỉnh số dòng hiển thị mỗi trang (10 / 20 / 30 / tùy chỉnh tối đa 50).
- Điều hướng phân trang với thông tin tổng số bản ghi.

Mỗi loại cảm biến được phân biệt bằng màu badge: đỏ cho Temperature, xanh dương cho Humidity, vàng cho Ambient Light.



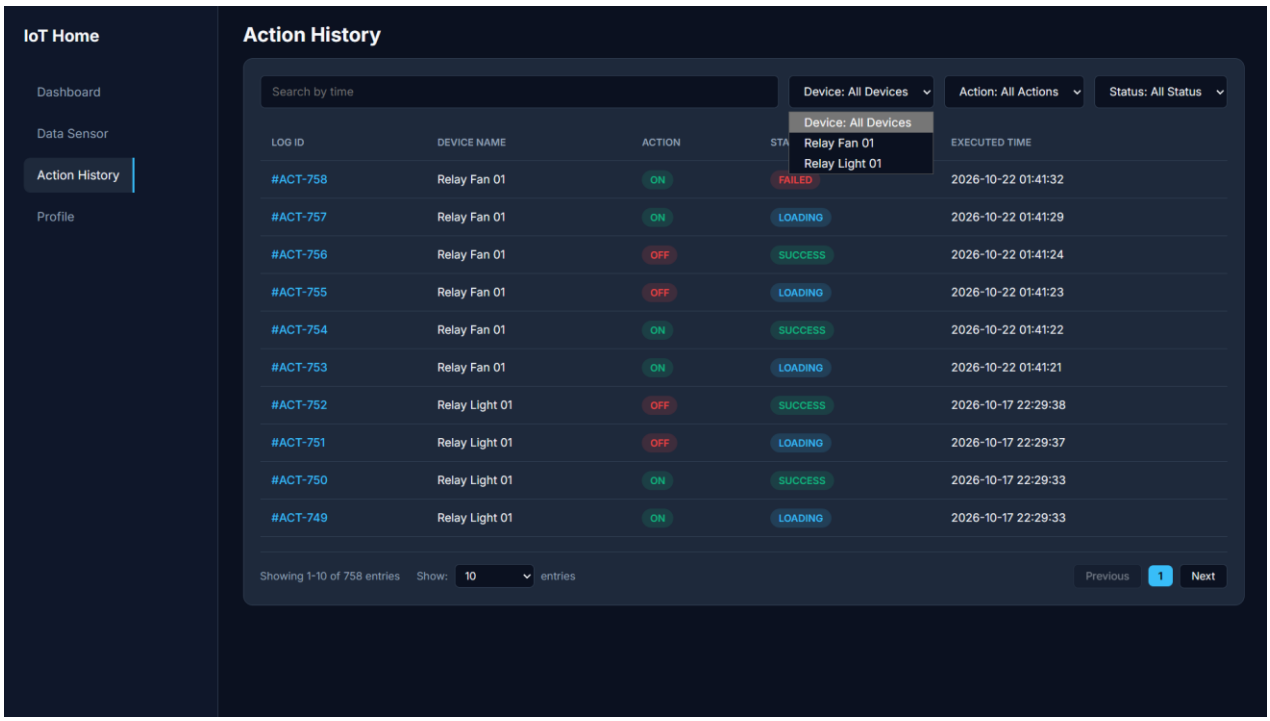
Hình 7 Giao diện Tab Data Sensor

6.3 Giao diện Tab Action History

Tab Action History cung cấp nhật ký đầy đủ các thao tác điều khiển thiết bị. Bộ lọc đa tiêu chí cho phép lọc đồng thời theo:

- Tên thiết bị (Relay Fan 01 / Relay Light 01)
- Loại hành động (ON / OFF)
- Trạng thái thực thi (SUCCESS / LOADING / FAILED)
- Tìm kiếm theo thời gian thực hiện

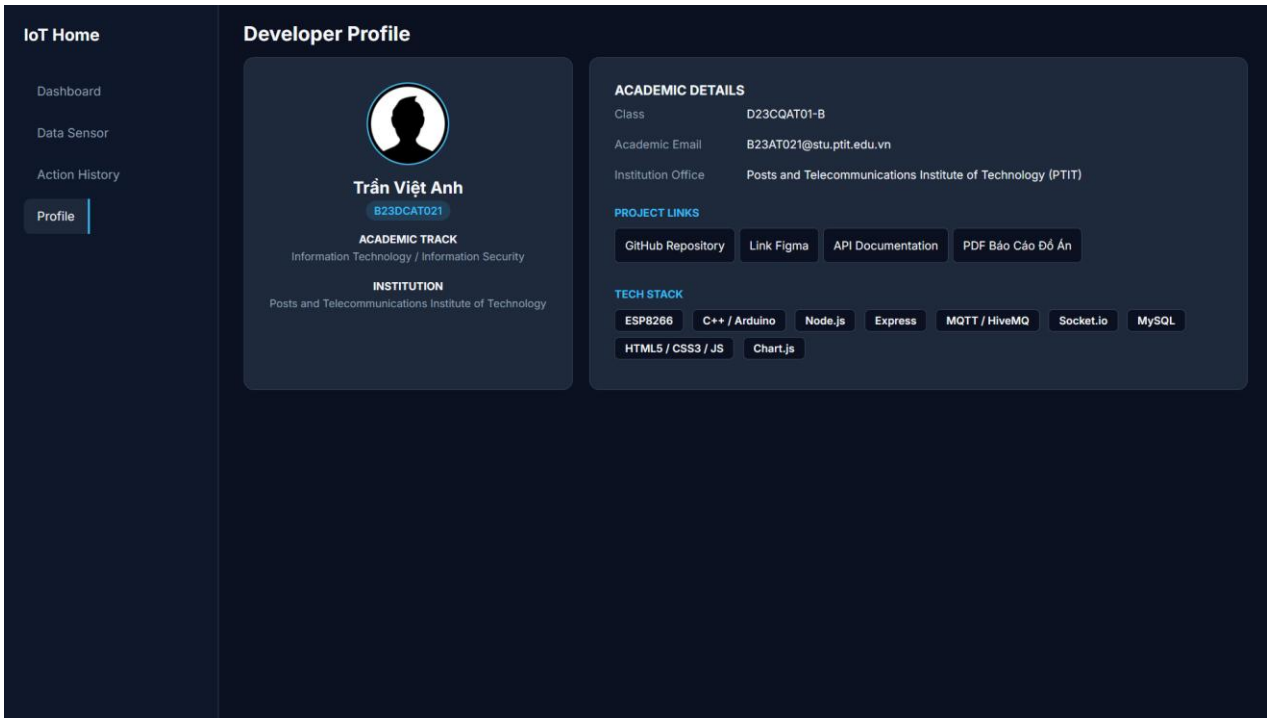
Trạng thái được hiển thị bằng badge màu phân biệt: xanh lá cho SUCCESS, xanh dương cho LOADING, đỏ cho FAILED.



Hình 8 Giao diện Tab Action History

6.4 Giao diện Tab Developer Profile

Tab Profile hiển thị thông tin định danh nhà phát triển bao gồm ảnh đại diện, tên, mã sinh viên, thông tin học thuật, các liên kết dự án (GitHub, Figma, Swagger, báo cáo PDF) và tech stack dưới dạng badge.



Hình 9 Giao diện Tab Developer Profile

6.5 Cơ chế Loading và phản hồi trạng thái

Khi người dùng kích hoạt công tắc relay, giao diện lập tức phản hồi bằng hiệu ứng spinner xoay trên thanh trượt toggle, trạng thái hiển thị chuyển sang “Status: ON

(Loading...)” và nút bị khóa để ngăn bấm trùng. Sau khi nhận phản hồi SUCCESS từ phần cứng, spinner biến mất và trạng thái cập nhật về giá trị cuối cùng. Nếu timeout sau 3.5 giây, công tắc tự động revert và Toast Notification đỏ xuất hiện ở góc trên phải thông báo lỗi.



Hình 10 Trạng thái Loading khi điều khiển thiết bị

6.6 Cảnh báo tự động ánh sáng thấp

Khi cường độ ánh sáng dưới 50 Lux, hệ thống đồng thời thực hiện ba hành động: hiển thị cảnh báo màu vàng cảnh báo trên giao diện, đánh dấu thẻ Ambient Light với viền đỏ nhấp nháy và tự động phát lệnh bật nếu đèn đang tắt và không có lệnh nào đang chờ xử lý.



Hình 11 Toast Notification cảnh báo ánh sáng thấp

CHƯƠNG 7. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

7.1 Kết quả đạt được

Qua quá trình nghiên cứu và triển khai, đề tài đã hoàn thành đầy đủ các mục tiêu đề ra ban đầu:

Về mặt kỹ thuật:

- Xây dựng thành công hệ thống IoT hoàn chỉnh từ phần cứng đến giao diện người dùng, tích hợp đồng thời ba giao thức truyền thông MQTT, WebSocket và RESTful HTTP.
- Triển khai cơ chế điều khiển thiết bị hai chiều với xác nhận kết quả (LOADING → SUCCESS/FAILED) và xử lý timeout tự động, đảm bảo tính nhất quán giữa trạng thái phần cứng và giao diện.
- Xây dựng cơ sở dữ liệu MySQL với cấu trúc chuẩn hóa, hỗ trợ lưu trữ và truy vấn hiệu quả dữ liệu cảm biến theo chuỗi thời gian.
- Tích hợp tài liệu API chuẩn OpenAPI 3.0, hỗ trợ kiểm thử trực tiếp các endpoint.

Về mặt chức năng:

- Giám sát môi trường thời gian thực với độ trễ dưới 500ms trong điều kiện mạng ổn định.
- Điều khiển từ xa hai thiết bị relay với phản hồi trạng thái chính xác.
- Tự động hóa bật đèn khi ánh sáng yếu dưới ngưỡng 50 Lux kèm cảnh báo trực quan.
- Tra cứu lịch sử dữ liệu cảm biến và nhật ký điều khiển với đầy đủ chức năng lọc, tìm kiếm và phân trang.

7.2 Hạn chế hiện tại

Bên cạnh những kết quả đạt được, hệ thống hiện còn một số hạn chế cần khắc phục trong các giai đoạn phát triển tiếp theo:

Bảo mật chưa đầy đủ: Hệ thống chưa triển khai xác thực người dùng (Authentication) và phân quyền (Authorization). currentUserId đang được hardcode là 1 trong mã nguồn, thông tin cấu hình nhạy cảm (mật khẩu DB) chưa được tách ra file môi trường .env.

MQTT chưa mã hóa: Hệ thống sử dụng HiveMQ public broker qua giao thức mqtt:// (không mã hóa), phù hợp cho môi trường học thuật nhưng chưa đáp ứng yêu cầu bảo mật cho môi trường production.

Chưa hỗ trợ đa người dùng: Kiến trúc hiện tại chỉ phục vụ một người dùng quản trị, chưa có cơ chế quản lý phiên làm việc hay phân quyền truy cập theo vai trò.

Dữ liệu cảm biến tăng trưởng không giới hạn: Bảng sensor_data sẽ tăng trưởng rất nhanh (3 bản ghi mỗi 2 giây ≈ 129.600 bản ghi/ngày) mà chưa có chiến lược Data Retention hay Data Archiving.

7.3 Hướng phát triển

Ngắn hạn:

- Triển khai Authentication/Authorization với JWT, tách thông tin cấu hình nhạy cảm ra file .env.
- Nâng cấp kết nối MQTT sang mqtts:// (TLS/SSL) để mã hóa toàn bộ luồng dữ liệu.
- Thêm chiến lược Data Retention tự động xóa hoặc lưu trữ dữ liệu cảm biến cũ.

Trung hạn:

- Mở rộng hệ thống cảm biến: bổ sung cảm biến khí CO2, chất lượng không khí (PM2.5), cảm biến chuyển động PIR.
- Phát triển tính năng lập lịch tự động (Scheduler) cho phép đặt lịch bật/tắt thiết bị theo thời gian biểu cố định.
- Xây dựng module cảnh báo nâng cao (thresholds tùy chỉnh, gửi email/Telegram notification).
- Tối ưu hiệu năng truy vấn CSDL với indexing và query caching.

Dài hạn:

- Tích hợp Machine Learning để dự đoán xu hướng môi trường và tối ưu hóa lịch vận hành thiết bị tự động.
- Phát triển ứng dụng di động (React Native/Flutter) để quản lý hệ thống qua smartphone.
- Triển khai lên Cloud (AWS IoT / Azure IoT Hub) để hỗ trợ quy mô lớn hơn với nhiều thiết bị và người dùng đồng thời.
- Áp dụng kiến trúc Microservices để tăng khả năng mở rộng và độ sẵn sàng của hệ thống.

TÀI LIỆU THAM KHẢO

[1] Espressif Systems, *ESP8266 Technical Reference*, Version 1.7, 2020. [Online]. Available: https://www.espressif.com/en/support/documents/technical-documents

[2] OASIS Standard, *MQTT Version 5.0*, OASIS Open, 2019. [Online]. Available: https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html

[3] HiveMQ, *HiveMQ MQTT Broker Documentation*, 2024. [Online]. Available: https://www.hivemq.com/docs/hivemq/4.x/user-guide/

[4] Node.js Foundation, *Node.js Documentation v18.x*, 2024. [Online]. Available: https://nodejs.org/docs/latest-v18.x/api/

[5] Express.js Team, *Express.js Guide and API Reference*, 2024. [Online]. Available: https://expressjs.com/en/guide/

[6] Socket.IO, *Socket.IO Documentation v4*, 2024. [Online]. Available: https://socket.io/docs/v4/

[7] MySQL AB, *MySQL 8.0 Reference Manual*, Oracle Corporation, 2024. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[8] OpenAPI Initiative, *OpenAPI Specification 3.0.3*, 2021. [Online]. Available: https://spec.openapis.org/oas/v3.0.3

[9] Chart.js Contributors, *Chart.js Documentation*, 2024. [Online]. Available: https://www.chartjs.org/docs/latest/

[10] Swagger, *Swagger UI Documentation*, SmartBear Software, 2024. [Online]. Available: https://swagger.io/tools/swagger-ui/